

FastAPI Template

FastAPI 项目模板，适合基于 FastAPI 构建高性能 Web 应用，开箱即用。

特性

- 异步架构 — 数据库驱动、ORM 会话、路由处理全异步，Uvicorn 多 Worker 水平扩展
- **FastAPI 特性实例** — 依赖注入（Depends）、Pydantic 数据校验与序列化、BackgroundTasks 后台任务、中间件、请求响应
- 统一响应格式 — `{code, msg, data}` 泛型包装，分页响应，预定义错误响应模型
- 全局异常处理 — 业务异常、参数校验、HTTP 异常统一拦截
- 请求日志 — IP（支持 X-Forwarded-For）、路径、参数、耗时、敏感参数脱敏
- **API 文档** — FastAPI 自动生成 Swagger UI，开发环境自动开启，生产环境关闭
- 质量控制 — 代码提交前通过 Ruff/Mypy 自动代码静态审查，代码提交后 CICD 流程进行单元测试、Docker Compose 部署验证，验证业务功能是否正常
- 错误追踪 — 使用 Sentry 进行系统错误追踪和性能监控，告警自动转发为飞书卡片消息
- 多环境流程 — 开发、预发布、生产，环境配置隔离，部署流程脚本化
- 容器化部署 — 多阶段镜像构建、Traefik 反代、数据库定时备份与迁移，deploy.sh 一键部署

环境要求

- Python 3.12+
- Docker & Docker Compose（生产部署）
- uv（包管理）

开发

1. 环境准备

```
# 前置要求: Python 3.12+、Docker、uv（包管理器）# 安装 uv:  
https://docs.astral.sh/uv/getting-started/installation/  
# 克隆项目  
git clone <repo-url> && cd fastapi-template
```

2. 启动数据库

使用 Docker 运行 PostgreSQL，数据通过 named volume 持久化：

```
cp .env.dev .env                # 同步开发环境配置
docker compose up -d            # 启动 PostgreSQL
docker compose ps                # 确认 db 状态为 healthy
```

3. 安装依赖

```
uv sync --all-extras            # 安装运行时 + 开发依赖 (pytest、ruff、mypy 等)
```

4. 数据库迁移

```
uv run alembic upgrade head      # 执行迁移，建表
# 新增/修改模型后，生成迁移文件
uv run alembic revision --autogenerate -m "add xxx table"
uv run alembic upgrade head
# 回滚
uv run alembic downgrade -1
```

5. 启动应用

```
uv run fastapi run app/main.py --reload # 开发模式，自动热重载
```

6. 验证

```
curl http://localhost:8000/health#
{"code":200,"msg":"ok","data":{"status":"ok"}}
```

开发环境访问 API 文档：<http://localhost:8000/docs>

7. 一键启动

```
bash dev.sh    # 自动完成：同步配置 → 安装依赖 → 启动数据库 → 迁移 → 启动服务
```

质量检查

单元测试

```
uv run pytest tests/ -v --cov --cov-report=term-missing# 覆盖率阈值 80%，  
低于此值测试失败# 测试使用 SQLite 内存数据库，无需启动 PostgreSQL
```

代码质量

```
# Ruff - lint + 格式化  
uv run ruff check .           # 代码检查  
uv run ruff format --check .  # 格式化检查  
# Mypy - 类型检查  
uv run mypy app/  
# Pre-commit - 提交前自动执行以上所有检查  
uv run pre-commit install     # 安装 git hooks (首次)  
uv run pre-commit run --all-files  # 手动全量检查
```

推送到 GitHub

推送代码或创建 PR 时，GitHub Actions 自动执行三项 CI 检查：

Workflow	检查内容
Lint	Ruff lint + 格式化检查 + Mypy 类型检查
Test Backend	pytest 单元测试 + 覆盖率报告 (≥80%)
Test Compose	Docker 构建镜像 → 完整部署 → 健康检查，验证部署流程

所有检查通过后方可合并 PR。

环境变量

提供三套环境配置文件：

文件	环境	说明
<code>.env.dev</code>	开发	本地环境运行
<code>.env.staging</code>	预发布	Docker Compose 方式运行
<code>.env.prod</code>	生产	Docker Compose 方式运行

变量说明：

变量	默认值	说明
<code>APP_ENV</code>	dev	环境（dev / staging / prod）
<code>LOG_LEVEL</code>	INFO	日志级别
<code>DB_DEBUG</code>	False	是否打印 SQL
<code>WORKERS</code>	4	Worker 进程数
<code>POSTGRES_SERVER</code>	localhost	数据库地址
<code>POSTGRES_PORT</code>	5432	数据库端口
<code>POSTGRES_USER</code>	postgres	数据库用户
<code>POSTGRES_PASSWORD</code>	postgres	数据库密码
<code>POSTGRES_DB</code>	fastapi_template	数据库名
<code>SECRET_KEY</code>		JWT 签名密钥
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	30	Access token 过期时间
<code>REFRESH_TOKEN_EXPIRE_DAYS</code>	7	Refresh token 过期时间
<code>SENTRY_DSN</code>		Sentry DSN
<code>SENTRY_SAMPLE_RATE</code>	0.1	Sentry 采样率
<code>FEISHU_WEBHOOK_URL</code>		飞书 Webhook 地址
<code>DOCKER_REGISTRY</code>		镜像仓库地址
<code>DOCKER_IMAGE</code>		镜像名
<code>DOCKER_TAG</code>		镜像版本

部署

镜像构建

Docker 镜像托管在阿里云 ACR(容器镜像服务)。在 `.env.prod` 中配置 `DOCKER_REGISTRY`、`DOCKER_IMAGE`、`DOCKER_TAG` (见环境变量表)，然后执行 `bash deploy.sh` 即可部署。

镜像打包有两种方式：

方式一：ACR 自动构建

向 GitHub 推送 `release-v$version` 格式的 tag 时，自动触发阿里云 ACR 构建镜像，镜像版本为 tag 中的 `$version` 部分：

```
git tag release-v0.2.0
git push origin release-v0.2.0# 构建完成后，更新 .env.prod 中的 DOCKER_TAG 并重新部署
```

方式二：手动构建推送

本地构建镜像并推送到阿里云 ACR：

```
docker build -t ${DOCKER_REGISTRY}/${DOCKER_IMAGE}:${DOCKER_TAG} .
docker push ${DOCKER_REGISTRY}/${DOCKER_IMAGE}:${DOCKER_TAG}
```

部署命令

```
# 部署生产环境
bash deploy.sh
# 部署预发布环境
bash deploy.sh staging
```

`deploy.sh` 执行流程：

1. 同步配置 (`.env.{env}` → `.env`)
2. 拉取最新镜像
3. 启动数据库
4. 备份数据库
5. 执行数据库迁移
6. 启动服务
7. 健康检查

服务模块

服务	说明
db	数据库
backend	FastAPI 应用，生产环境使用阿里云 ACR 镜像
traefik	反向代理，监听 80 端口，将请求转发到 backend
db_migrate	数据库迁移，使用 <code>migrate</code> profile，deploy.sh 中通过 <code>\$DC run --rm db_migrate</code> 一次性执行
db_backup	数据库定时备份，基于 <code>crond</code> 每日自动备份到 <code>db_backup/</code> 目录